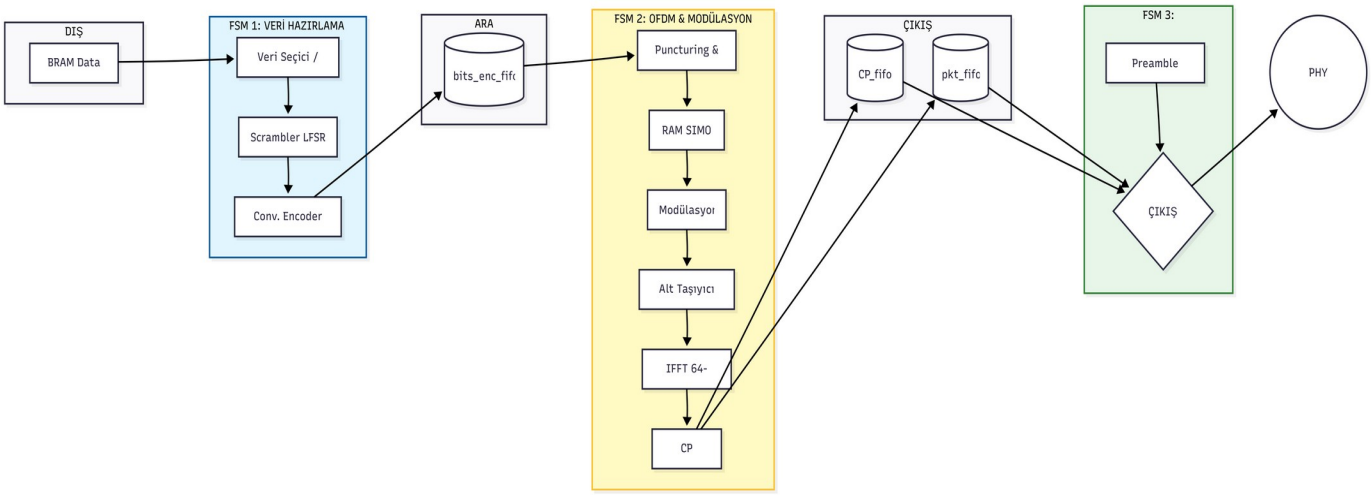
OpenOFDM Tx Çalışma Mantığı ve İç Yapısı

Table of Contents

1.OpenOFDM Tx İç Yapısı....3
A. FSM 1 Ham Veriyi Göndermeye Hazır Hale Getirme....3
1. Ana Durum Akışı (state1)....3
A. S1_WAIT_PKT (Bekleme)....3
B. S1_L_SIG (Legacy Signal Field)....3
C. S1_HT_SIG (High Throughput Signal Field)....4
D. S1_DATA (Veri Alanı - En Karmaşık Kısım)....4
2. Veri Alt Durumları (state11)....4
1. S11_SERVICE (Servis Alanı)....4
2. S11_PSDU_DATA (Kullanıcı Verisi)....4
3. S11_PSDU_CRC (Hata Kontrol)....5
4. S11_TAIL (Kuyruk Bitleri)....5
5. S11_PAD (Dolgu Bitleri)....5
FSM 2 Durum Akışı ve Detayları....5
1. S2_PUNC_INTERLV (Delikleme ve Aralama)....5
2. S2_PILOT_DC_SB (Pilot ve Hazırlık)....6
3. S2_MOD_IFFT_INPUT (Modülasyon ve IFFT Besleme)....6
4. S2_RESET (Sembol Sonu Kontrolü)....7
FSM 3 Durum Akışı ve Detayları....7
1. S3_WAIT_PKT (Başlama Vuruşu)....7
2. Preamble (Ön-Kod) Durumları (S3_L_STF, S3_L_LTF, S3_HT_...)....7
3. Sinyal ve Veri Durumları (S3_L_SIG, S3_HT_SIG, S3_DATA)....8
Kritik Mekanizma: Çıkış MUX'u ve FIFO Sıralaması....8
A. Çıkış Multiplexer'ı (MUX)....8
B. FIFO Turn (Veri vs. Devresel Önek)....8
FSM 1 Kod açıklamaları ve MATLAB simülasyonları....9
Veri Seçici....9
Scrambler....11
MATLAB Modeli:....12
Çıktısı:....14
Convolutinal Encoder....15
MATLAB Modeli....15
Çıktısı:....16

1.OpenOFDM Tx İç Yapısı



OpenOFDM Tx 3 farklı FSMden oluşur. Bu FSMler sırasıyla farklı görevleri yerine getiren bir pipeline oluşturur. Amaç ilk alınan headera göre RAMden okunan byteları uygun I ve Q verilerine dönüştürmektir.

A. FSM 1 Ham Veriyi Göndermeye Hazır Hale Getirme

Bu FSM iki farklı State Register içerir. Bunlar:

Ana Durumlar (`state1`): Paketin ana bölümlerini (Başlık vs. Gövde) yönetir.

Veri Alt Durumları (`state11`): Gövde (Body) kısmının içindeki detaylı alanları yönetir.

1. Ana Durum Akışı (`state1`)

Bu makine `phy_tx_start` sinyali geldiğinde uyanır ve şu sırayı takip eder:

A. S1_WAIT_PKT (Bekleme)

- **Görevi:** Sistem boştadır, başlatma komutu bekler.
- **Olay:** `phy_tx_start == 1` olduğunda `bram_addr` sıfırlanır ve `S1_L_SIG` durumuna geçilir.

B. S1_L_SIG (Legacy Signal Field)

- **Süre:** 24 bit (bit 0'dan 23'e kadar).
- **Görevi:** Paketin hızını ve uzunluğunu belirleyen kritik başlık bilgisini okur.
- **İşlem:**
    - BRAM'den gelen ilk 24 bit okunur.
    - **Parametre Çıkarımı:** Kodun şu kısmında (`case(bram_din[3:0])`) paketin hızı (6 Mbps, 54 Mbps vb.) belirlenir. Bu, `N_BPSC` (sembol başına bit) ve `N_DBPS` (OFDM sembolü başına veri biti) değişkenlerini ayarlar.

    - **Paket Türü:** `PKT_TYPE` (Legacy mi HT mi?) belirlenir.
    - **Önemli Not:** Bu alandaki bitler **Karıştırılmaz (Scramble edilmez)**, doğrudan kodlayıcıya gider.
- **Geçiş:** 24. bit işlendikten sonra, eğer paket HT (High Throughput) ise `S1_HT_SIG`'e, değilse doğrudan `S1_DATA`'ya geçilir.

C. S1_HT_SIG (High Throughput Signal Field)

- **Süre:** 48 bit (Eğer paket 802.11n/HT ise).
- **Görevi:** HT moduna özel (MCS, Aggregation, Short GI) parametreleri okur.
- **İşlem:** BRAM'den okunan verilerle `PSDU_BIT_LEN` (toplam veri uzunluğu) güncellenir.
- **Geçiş:** 48 bit bittiğinde `S1_DATA`'ya geçilir.

D. S1_DATA (Veri Alanı - En Karmaşık Kısım)

- Bu ana durum, paketin asıl yükünü taşır. Kendi içinde `state11` isimli bir alt makine çalıştırır.

2. Veri Alt Durumları (`state11`)

`state1` = `S1_DATA` olduğunda devreye girer ve paketin gövdesini oluşturur.

1. S11_SERVICE (Servis Alanı)

- **Uzunluk:** 16 bit.
- **Veri:** Tamamen '0' gönderilir.
- **Amaç:** Bu 16 bitlik sıfır dizisi, alıcı taraftaki karıştırıcının (descrambler) senkronize olmasını sağlar.
- **Mekanizma:** Veri BRAM'den okunmaz, kod içinde sabit sıfır olarak üretilir. Ancak karıştırıcı (scrambler) burada çalışmaya başlar ve `init_data_scram_state` yüklenir.

2. S11_PSDU_DATA (Kullanıcı Verisi)

- **Uzunluk:** `PSDU_BIT_LEN` kadar.
- **Görevi:** BRAM'den asıl veriyi (örneğin internet paketi, video verisi) çeker.
- **BRAM Yönetimi:**
    - `psdu_bit_cnt` her saat darbesinde artar.
    - 64 bitlik bir kelime bittiğinde (`psdu_bit_cnt[5:0] == 111111`), `bram_addr` bir arttırılır.
    - BRAM'den gelen 64 bit, `bram_din_last_nibble` gibi tamponlara alınarak hizalanır.
- **CRC Hesabı:** Bu aşamada akan her bit aynı zamanda `crc32_tx` modülüne gönderilir ve arka planda CRC hesaplanır.

3. S11_PSDU_CRC (Hata Kontrol)

- **Uzunluk:** 32 bit (4 Byte).
- **Görevi:** Veri bittikten hemen sonra, hesaplanan CRC değeri (`pkt_fcs`) veri akışına eklenir.
- **Kaynak:** BRAM değil, `crc32_tx` modülünün çıkışıdır.

4. S11_TAIL (Kuyruk Bitleri)

- **Uzunluk:** 6 bit.
- **Veri:** Tamamen '0'.
- **Amaç:** Bükme kodlayıcısının (Convolutional Encoder) iç durumunu (state) sıfıra döndürmek. Bu, paketin sonunda kod çözücünün doğru karar vermesini sağlar.
- **Karıştırma:** Bu bitler karıştırılmaz (bazı standartlarda farklılık gösterebilir, bu kodda: `bit_scram = 0` yapılmış, scrambler by-pass edilmiş).

5. S11_PAD (Dolgu Bitleri)

- **Uzunluk:** Değişken (OFDM sembolünü doldurana kadar).
- **Amaç:** 802.11'de veriler OFDM sembol blokları halinde gönderilir. Eğer veriniz son sembolü tam doldurmuyorsa, kalan kısım "Pad" bitleriyle doldurulur.
- **Mantık:** `dbps_cnt_FSM1` sayacı, o anki OFDM sembolünün dolup dolmadığını kontrol eder. Sayaç `N_DBPS` değerine ulaşana kadar rastgele (scrambled 0) bitler gönderilir.

FSM 2 Durum Akışı ve Detayları

FSM 2, `state2` kaydında tutulan 4 ana durumdan oluşur.

1. S2_PUNC_INTERLV (Delikleme ve Aralama)

Bu, verinin hazırlandığı ilk aşamadır.

- **Görevi:** FSM 1'den gelen kodlanmış bitleri (`bits_enc_fifo`) alır, gereksiz bitleri atar (Puncturing) ve kalanları belleğe karmaşık bir sırayla yazar (Interleaving).
- **Çalışma Mantığı:**
  - **Giriş:** `bits_enc_fifo`'dan 2 bit okur.
  - **LUT (Look-Up Table):** `punc_interlv_lut` modülüne o anki veri hızı (`RATE`) ve bit sırası (`dbps_cnt_FSM2`) gönderilir. LUT iki şey söyler:
    1. `punc_info`: "Bu bitleri atayım mı, saklayayım mı?"
    2. `interlv_addrs`: "Bu biti belleğin (`bits_ram`) hangi adresine yazayım?"
  - **RAM'e Yazma:** Eğer bit deliklenmeyecekse (punctured değilse), `bits_ram` modülüne LUT'tan gelen adrese yazılır.

- **Amaç:** Bitlerin yerini karıştırarak, kablosuz kanalda oluşabilecek toplu hataların (burst errors) veriyi yok etmesini engellemektir.
- **Bitiş:** Bir OFDM sembolünü dolduracak kadar bit (örneğin 48, 96, 192 veya 288 bit) işlendiğinde (`dbps_cnt_FSM2 == dbps_size-1`), bir sonraki duruma geçilir.

2. S2_PILOT_DC_SB (Pilot ve Hazırlık)

Veri bitleri RAM'e yazıldıktan sonra, frekans alanındaki diğer bileşenler hazırlanır.

- **Pilot Hazırlığı:** OFDM'de alıcının kanalı takip edebilmesi için veri arasına "Pilot" sinyalleri serpiştirilir.
    - Bu durum, 4 adet pilot alt taşıyıcısı için (`pilot_iq[0..3]`) değerleri (1 veya -1) hesaplar.
    - **Polarite:** `ht_polarity` ve `pilot_scram_state` (LFSR) kullanılarak pilotların işareti her sembolde değiştirilir. Bu, sinyalin tepri (PAPR) özelliğini iyileştirir.
- **IFFT Hazırlığı:** `ifft_ce <= 1` yapılarak IFFT modülü (Ters Fourier Dönüşümü) aktif hale getirilir.
- **Sayaç Sıfırlama:** IFFT giriş sayacı (`iq_cnt`) sıfırlanır.

3. S2_MOD_IFFT_INPUT (Modülasyon ve IFFT Besleme)

Bu, FSM 2'nin en kritik döngüsüdür. 0'dan 63'e kadar sayarak (`iq_cnt`), frekans alanındaki 64 alt taşıyıcıyı (subcarrier) tek tek IFFT girişine besler.

Burada **Frekans Haritalaması (Subcarrier Mapping)** yapılır. Her `iq_cnt` değeri için `ifft_iq` şu şekilde seçilir:

iq_cnt (0-63)	Alt Taşıyıcı Tipi	Kaynak	Açıklama
0	DC	`DC_SB_IQ`	Merkez frekans (0 Hz), boş bırakılır.
1 - 5	Guard (Yan Bant)	`DC_SB_IQ`	Girişimi önlemek için boş bırakılır.
7, 21, 43, 57	Pilot	`pilot_iq[]`	Kanal tahmini için pilot sinyalleri.
27 - 37	DC & Guard	`DC_SB_IQ`	Spektrumun ortasındaki boşluk (DC etrafı).
Diğerleri	**VERİ**	`mod_IQ`	`bits_ram`'den okunup modüle edilen veri.

Modülasyon İşlemi: Veri alt taşıyıcısı sırası geldiğinde;

1. FSM 2, `iq_cnt` değerine göre `bits_ram`'den hangi adresin okunacağını (`mod_addr`) hesaplar.
2. RAM'den okunan bitler `modulation` bloğuna girer.
3. Modülasyon bloğu bitleri alır (örn: "11") ve bunu bir karmaşık sayıya (örn: QPSK için 0.707 + j0.707) çevirir.
4. Bu değer `ifft_iq` olarak atanır.

- **HT vs Legacy:** Kodda `if(PKT_TYPE == HT ...)` blokları göreceksiniz. 802.11n (HT) standardı, 802.11a/g (Legacy) standardına göre biraz daha fazla alt taşıyıcı kullanır. Kod bu ayrımı burada yönetir.

4. S2_RESET (Sembol Sonu Kontrolü)

Bir OFDM sembolünün 64 örneği de IFFT'ye gönderildikten sonra bu duruma gelinir.

- **Sembol Sayacı:** `ofdm_cnt_FSM2` bir arttırılır.
- **Döngü Kontrolü:**
  - Eğer işlenecek daha fazla sembol varsa (`ofdm_cnt_FSM2 < ofdm_cnt_FSM1`), tekrar başa (`S2_PUNC_INTERLV`) dönülür.
  - Tüm paket bittiyse, FSM burada bekler veya durur.
- **IFFT Kapatma:** `ifft_ce` (Clock Enable) kontrol edilerek IFFT'nin gereksiz çalışması önlenir.

FSM 3 Durum Akışı ve Detayları

FSM 3, 802.11 paket yapısını (Frame Structure) sırasıyla takip eden durumlardan oluşur.

1. S3_WAIT_PKT (Başlama Vuruşu)

- **Durum:** Sistem boştadır.
- **Tetik:** `phy_tx_start` sinyali geldiğinde "Start" verilir.
- **İşlem:**
  - `preamble_addr` (ROM okuma adresi) sıfırlanır.
  - `S3_L_STF` durumuna geçilir.

2. Preamble (Ön-Kod) Durumları (S3_L_STF, S3_L_LTF, S3_HT_...)

Bu aşamalar, paketin en başındaki sabit, standart sinyalleri gönderir. Bu sinyaller dinamik olarak üretilmez, önceden kaydedilmiş ROM belleklerinden (`l_stf_rom`, `l_ltf_rom`) okunur.

- **S3_L_STF (Legacy Short Training Field):**
  - **Amaç:** Alıcının sinyalin varlığını algılaması (Packet Detection) ve otomatik kazanç kontrolü (AGC) yapması içindir.
  - **İşleyiş:** `preamble_addr` 0'dan 159'a kadar sayar (Toplam 160 örnek). `phy_tx_started` sinyali burada '1' yapılır.
- **S3_L_LTF (Legacy Long Training Field):**
  - **Amaç:** Alıcının kanal denkleştirmesi (Channel Estimation) yapması içindir.
  - **İşleyiş:** Adres tekrar 0-159 arası sayar ve ROM'dan okunan veriyi çıkışa verir.
- **HT Preamble (S3_HT_STF, S3_HT_LTF):**

- Eğer paket Yüksek Hızlı (HT/802.11n) bir paketse, Legacy preamble'dan sonra bu durumlar devreye girer ve HT'ye özel eğitim sinyallerini yine ilgili ROM'lardan okuyarak gönderir.

3. Sinyal ve Veri Durumları (S3_L_SIG, S3_HT_SIG, S3_DATA)

Preamble bittiğinde, FSM 3 artık ROM'ları bırakır ve FSM 2'nin doldurduğu FIFO'lardan (`pkt_fifo` ve `CP_fifo`) veri çekmeye başlar.

- **Bu aşamada FSM 3'ün mantığı değişir:** Artık bir adres sayacı değil, gönderilen toplam örnek sayısını (`pkt_iq_sent` ve `CP_iq_sent`) takip eder.
- **S3_L_SIG & S3_HT_SIG:**
  - Paketin hızını ve boyunu içeren başlık bilgileridir (Signal Field). Bunlar da aslında birer OFDM sembolüdür. FSM 3 bu aşamada FIFO'dan veriyi çekerken, doğru sayıda örnek gönderildiğinden emin olur.
- **S3_DATA (Ana Veri Gövdesi):**
  - Paketin en uzun süren kısmıdır.
  - **Bitiş Kontrolü:** Gönderilen toplam örnek sayısı (`pkt_iq_sent + CP_iq_sent`), hesaplanan toplam paket uzunluğuna (`nof_iq2send`) ulaştığında paket bitmiş demektir.
  - **Sonuç:** `phy_tx_done` sinyali '1' yapılır ve `FSM3_reset` tetiklenerek tüm sistem bir sonraki paket için sıfırlanır.

Kritik Mekanizma: Çıkış MUX'u ve FIFO Sıralaması

FSM 3'ü anlamak için kodun en altındaki `assign` bloklarına ve `fifo_turn` mantığına bakmak gerekir. Burası bir tren rayı makası (switch) gibi çalışır.

A. Çıkış Multiplexer'ı (MUX)

Verilog kodunun sonundaki şu satır, o an dışarıya hangi verinin gideceğini seçer:

Verilog

```
assign result_i =
      state3 == S3_L_STF ? l_stf[31:16] :  // Durum STF ise -> STF ROM'dan oku
     (state3 == S3_L_LTF ? l_ltf[31:16] :  // Durum LTF ise -> LTF ROM'dan oku
     ...
     (fifo_turn == PKT_FIFO ? pkt_fifo_odata[31:16] : CP_fifo_odata[31:16])); //
Değilse -> FIFO'dan oku
```

Bu tek satırlık kod, FSM 3'ün o anki durumuna göre veriyi nereden yönlendireceğine karar verir.

B. FIFO Turn (Veri vs. Devresel Önek)

OFDM'de her sembolün başına bir "Koruma Aralığı" (Devresel Önek - CP) eklenir. FSM 3 veri gönderirken (`S3_DATA`), `fifo_turn` isimli paralel bir mantık sürekli çalışır:

1. **Sıra CP'de (`fifo_turn == CP_FIFO`):** Önce `CP_fifo`'dan 16 adet (veya Short GI için 8) örnek gönderilir.
2. **Sıra Veride (`fifo_turn == PKT_FIFO`):** CP bitince makas değişir, `pkt_fifo`'dan 64 adet ana sembol örneği gönderilir.
3. **Döngü:** Bu işlem paket bitene kadar her OFDM sembolü için tekrarlanır (CP -> Data -> CP -> Data...).

FSM 1 Kod açıklamaları ve MATLAB simülasyonları

Veri Seçici

İletim, *phy_tx_start* sinyali ile başlatılır ve ilk FSM'in görevi iletim modunu belirleyen preamble bilgilerini BRAM'den okuyarak iletimi buna göre organize etmektir. Sistem, 64-bitlik BRAM yapısından veri çekecek şekilde tasarlanmıştır ve BRAM'in ilk iki adresi, iletimi konfigüre eden header/preamble bilgileri için rezerve edilmiştir.

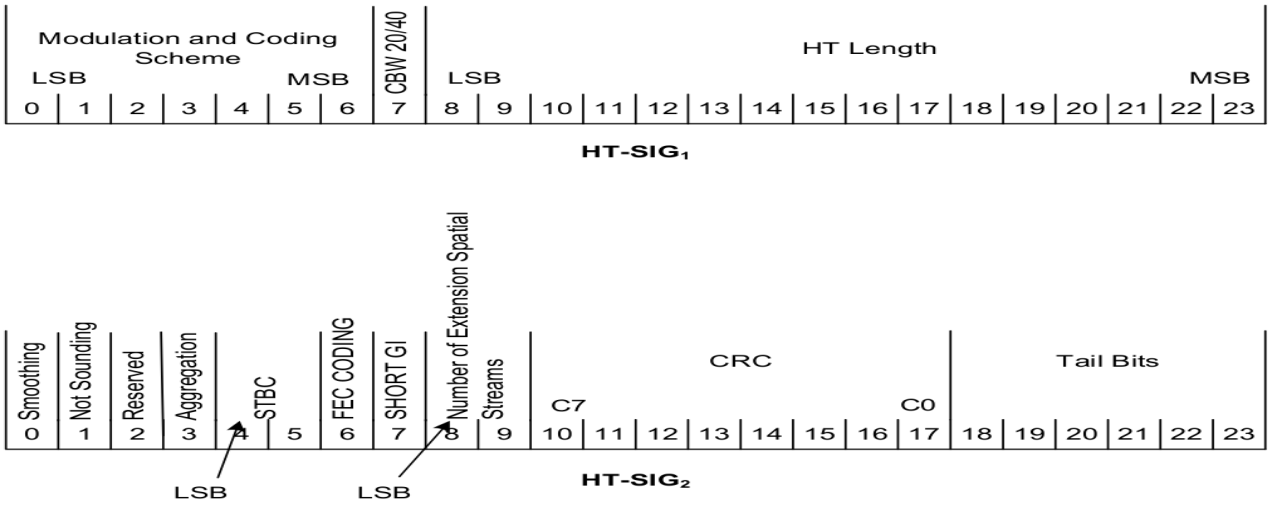
IEEE 802.11n (Wi-Fi 4) standardında iki farklı preamble yapısı tanımlanmıştır:

- Eski cihazlarla uyumluluk için Legacy (SIGNAL) preamble,
- Yeni nesil cihazlar için HT (High Throughput) preamble.

Bu Tx modülü her iki preamble türüyle de konfigüre edilip iletim yapabilecek şekilde tasarlanmıştır. HT-SIGNAL, Legacy SIGNAL yapısının üzerine inşa edilmiştir. Legacy SIGNAL alanı 24 bit uzunluğundadır ve 24. bit, preamble'ın Legacy mi yoksa HT mi olduğunu belirler. Eğer bu bit 1 ise preamble HT-SIGNAL olarak yorumlanır ve HT-SIGNAL bilgileri için BRAM'in 1. adresi kullanılır.

OpenOFDM tasarımı yalnızca mixed format (Legacy SIGNAL + HT-SIGNAL) desteği sunmaktadır. Kullanılacak sistemde her iki taraf da 802.11n proHeading 3tokolünü takip edeceği için HT-SIGNAL kullanılabilir. Bunun için Legacy SIGNAL alanında Rate değeri 6 Mbps olacak şekilde ilk 4 bit 1011 olarak ayarlanmalı ve 24. bit 1 yapılmalıdır. Alıcı bu bilgiyi okuduğunda kendisini HT-SIGNAL moduna geçirerek iletime devam edecektir.

HT-SIGNAL için preamble 48 bit olarak tanımlanmış ve şu sekilde rezerve edilmiştir:



Burada HT-SIG1 ilk 24 biti(0-23) temsil ederken HT-SIG2 sonraki 24 biti (24-47) ifade eder. Ilk 7 bit (0-6) modulasyon tipi ve coding rate belirler (MCS standartında). OpenOFDM sisteminde sadece 0-7 arası destekler. Bunlar:

Modulation and coding schemes

MCS index	Spatial streams	Modulation type	Coding rate	Data rate (Mbit/s)[b]			
				20 MHz channel		40 MHz channel	
				800 ns GI	400 ns GI	800 ns GI	400 ns GI
0	1	BPSK	1/2	6.5	7.2	13.5	15
1	1	QPSK	1/2	13	14.4	27	30
2	1	QPSK	3/4	19.5	21.7	40.5	45
3	1	16-QAM	1/2	26	28.9	54	60
4	1	16-QAM	3/4	39	43.3	81	90
5	1	64-QAM	2/3	52	57.8	108	120
6	1	64-QAM	3/4	58.5	65	121.5	135
7	1	64-QAM	5/6	65	72.2	135	150

8.bit (7) Channel Bandwithini ifade eder ve OpenOFDM için sadece 0 olmalıdır (0 20MHz ifade eder 1 ise 40 MHz) kullanılabilir. Geri kalan (8-23 arası)gönderilen data uzunluğunu ifade eder(bit cinsinden).
HT-SIG2'nin 0-6 bitleri arası(BRAMde 24-30 arası) OpenOFDM standartlarına göre 0100000 olarak ayarlanmalı. 7.bit(BRAMde 31) Short GI(Guard Interval) kullanılıp kullanılmayacağını belirler. OpenOFDM ikisinide hem uzun GI hemde kısa GI destekler . Yine 8. ve 9. bitler (BRAM 32 ve 33) 0 olmalı.10-17 (BRAM 34-41) arası önceki 34 bitin CRC'si (Cyclic Redundancy Check) yazılır (datanın errorsuz aktarıldığının test edilmesi için) .Son 6 bit ise kuyruk bitidir ve 0 yazılır .HT-SIG2 hakkında bilgiler buradan ulaşılabilir .

Kod üzerinde 24.bit okunduktan sonra eğer Legacy SIGNAL 6 Mbps ve 24.bit 1 olarak belirtdiyse FSM1 state1 HT-SIG durumuna geçer ve burada BRAM'ın 1. adresini okur. :

```
// HT packet
if(RATE == 5'b01011 && PKT_TYPE == HT) begin
PKT_TYPE <= HT;
state1 <= S1_HT_SIG;
```

sonrasında ise BRAMden okuduğu adresin ilk 3 bitine göre uygulanacak kodlama ve modülasyon tipine karar verir :

```
S1_HT_SIG: begin
    plcp_bit_cnt <= plcp_bit_cnt + 1;
    if(plcp_bit_cnt == 0) begin
        case(bram_din[2:0])
            3'b000:  begin  N_BPSC <= 1;  N_DBPS <= 26;  RATE <= 5'b10000; end  //  6.5 Mbps MCS0
            3'b001:  begin  N_BPSC <= 2;  N_DBPS <= 52;  RATE <= 5'b10001; end  // 13.0 Mbps MCS1
            3'b010:  begin  N_BPSC <= 2;  N_DBPS <= 78;  RATE <= 5'b10010; end  // 19.5 Mbps MCS2
            3'b011:  begin  N_BPSC <= 4;  N_DBPS <= 104; RATE <= 5'b10011; end  // 26.0 Mbps MCS3
            3'b100:  begin  N_BPSC <= 4;  N_DBPS <= 156; RATE <= 5'b10100; end  // 39.0 Mbps MCS4
            3'b101:  begin  N_BPSC <= 6;  N_DBPS <= 208; RATE <= 5'b10101; end  // 52.0 Mbps MCS5
            3'b110:  begin  N_BPSC <= 6;  N_DBPS <= 234; RATE <= 5'b10110; end  // 58.5 Mbps MCS6
            3'b111:  begin  N_BPSC <= 6;  N_DBPS <= 260; RATE <= 5'b10111; end  // 65.0 Mbps MCS7
            default: begin  N_BPSC <= 1;  N_DBPS <= 26;  RATE <= 5'b10000; end  //  6.5 Mbps
        endcase
        PSDU_BIT_LEN <= {bram_din[23:8], 3'd0};
        HT_AGGR <= bram_din[27];
        S_GI <= bram_din[31];
    end else if(plcp_bit_cnt == 23) begin
        ofdm_cnt_FSM1 <= ofdm_cnt_FSM1 + 1;

    end else if(plcp_bit_cnt == 47) begin
        ofdm_cnt_FSM1 <= ofdm_cnt_FSM1 + 1;

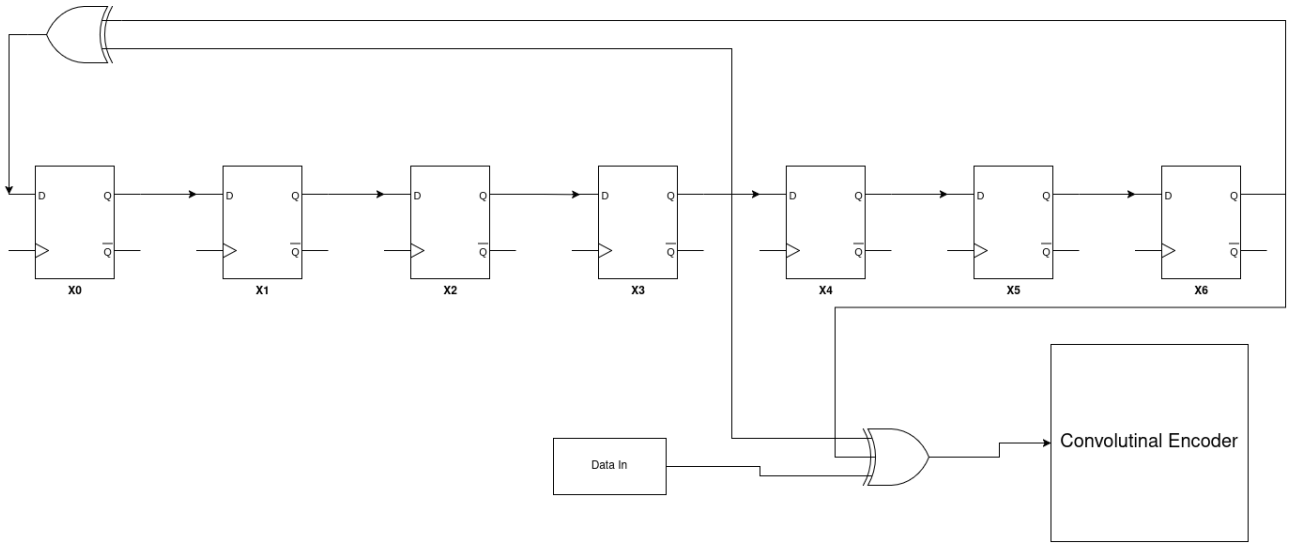
        service_bit_cnt <= 0;
        data_scram_state <= init_data_scram_state;
        dbps_cnt_FSM1 <= 0;

        state1 <= S1_DATA;
        state11 <= S11_SERVICE;
    end
end
```

Burada belirtilen MCS koduna göre FSM2 de kullanılacak olan N_BPSC(Bit Per Sub-Carier), N_DBPS(Data Bits Per Symbol),PSDU_Bit_LEN,HT_AGR,S_GI ve RATE belirlenir. Rate ve N_DBPS FSM2 de code rate ayarlarken N_BPSC modülasyon tipini ayarlar. PSDU_BIT_LEN datanın uzunluğunu ayarlar . Preamble okunduktan sonra FSM1 S1_Data durumuna geçer. Bu noktada Scrambler ve Convolutinal Encoder devreye girer ve Data bitlerin akışı başlar.

Scrambler

OpenOFDM Scrambler bloğu bir adet 7 bitlik LFSR içerir ve bu LFSRyi kullanarak gönderdiğimiz Datadaki oluşabilecek uzun 1 ve 0 serilerinin önüne geçer. Öncelikle bu LFSRnin seedi dışardan init_data_scram_state ile verilir. FSM1 S1_DATA durumuna geçiş yapınca bu LFSR aktif hale gelir. 802.11 Standartında bu LFSR için 16 clock cycle boyunca service biti göndermesi gerekir bunu S11_SERVICE durumunda gerçekleşir.Bu durum boyunca LFSRnin 7. ve 4. bitleri 0 (XOR işlemi için etkisiz) ile XOR işlemine girer. Bu Rx tarafında desramber işlemi için gereklidir. Bu işlemden sonra state_11 S11_PSDU_DATA işlemine geçer. Sonrasında ise okunan Datalar LFSR'nin 7. ve 4. biti ile XOR işlemine girer.



MATLAB Modeli:

```
clc; clear; close all;

%% 1. PARAMETRELER VE VERİ OLUŞTURMA
% Verilog'daki 'init_data_scram_state' (Başlangıç Tohumu - 7 bit)
% Not: Hepsi sıfır olmamalıdır.
init_seed = [1 0 1 1 1 0 1];

% Daha fazla veri biti (Örneğin 64 bitlik rastgele veri)
packet_length = 64;
input_data = randi([0 1], 1, packet_length);

% Verilog'daki 'S11_SERVICE' durumu: 16 adet '0' biti
service_field = zeros(1, 16);

% Toplam İşlenecek Paket: [Service Field (16 bit)] + [Data (64 bit)]
full_payload = [service_field, input_data];

%% 2. TX SCRAMBLER SİMÜLASYONU
% Polinom: x^7 + x^4 + 1
% Mantık: Feedback = State[6] ^ State[3]

current_state = init_seed;
tx_output = zeros(1, length(full_payload));

% Sonuçları görselleştirmek için string değişkenler
str_input = '';
str_scrambled = '';
```

```
fprintf('--- TX SCRAMBLER BAŞLIYOR ---\n');
fprintf('Başlangıç State (Seed): %s\n\n', num2str(init_seed));

for k = 1:length(full_payload)
    % Verilog: data_scram_state[6] (Matlab index 1) ve [3] (Matlab index 4)
    b6 = current_state(1);
    b3 = current_state(4);

    % Feedback (XOR İşlemi)
    feedback = xor(b6, b3);

    % Çıkış Biti Hesaplama
    % Verilog: bit_scram = state[6] ^ state[3] ^ input
    % Not: Service kısmında input zaten 0 olduğu için sadece feedback çıkar.
    input_bit = full_payload(k);
    scrambled_bit = xor(input_bit, feedback);

    % Sonucu kaydet
    tx_output(k) = scrambled_bit;

    % Debug Çıktısı (İsteğe bağlı, her adımı görmek için)
    % if k <= 20 % İlk 20 biti detaylı yazdır
    %    fprintf('Bit %d: In=%d | State=%s | Out=%d\n', ...
    %        k, input_bit, num2str(current_state), scrambled_bit);
    % end

    % State Güncelleme (Shift Register - Sola Kaydır)
    % Verilog: {data_scram_state[5:0], feedback}
    current_state = [current_state(2:end), feedback];
end

%% 3. SONUÇLARI GÖSTER
% Verileri gruplayarak gösterelim

scrambled_service = tx_output(1:16);
scrambled_data    = tx_output(17:end);

disp('------------------------------------------------------------');
disp('GİRİŞ VERİSİ (INPUT DATA - İlk 64 Bit):');
disp(num2str(input_data));
disp('------------------------------------------------------------');
disp(' ');
disp('ÇIKIŞ - BÖLÜM 1: SERVICE FIELD (İlk 16 Bit - Input hep 0 idi):');
disp(num2str(scrambled_service));
disp(' ');
```

```
disp('ÇIKIŞ - BÖLÜM 2: SCRAMBLED DATA (Karıştırılmış Veri):');
disp(num2str(scrambled_data));
disp('------------------------------------------------------------');

% Grafiksel Gösterim (Bitlerin dağılımını görmek için)
figure;
subplot(2,1,1);
stem(input_data, 'filled');
title('Orijinal Veri Girişi');
ylim([-0.5 1.5]); xlim([0 packet_length]);
grid on;

subplot(2,1,2);
stem(scrambled_data, 'filled', 'r');
title('Scramble Edilmiş Veri Çıkışı');
ylim([-0.5 1.5]); xlim([0 packet_length]);
grid on;
```

Çıktısı:

--- TX SCRAMBLER BAŞLIYOR ---

Başlangıç State (Seed): 1 0 1 1 1 0 1

GİRİŞ VERİSİ (INPUT DATA - İlk 64 Bit):

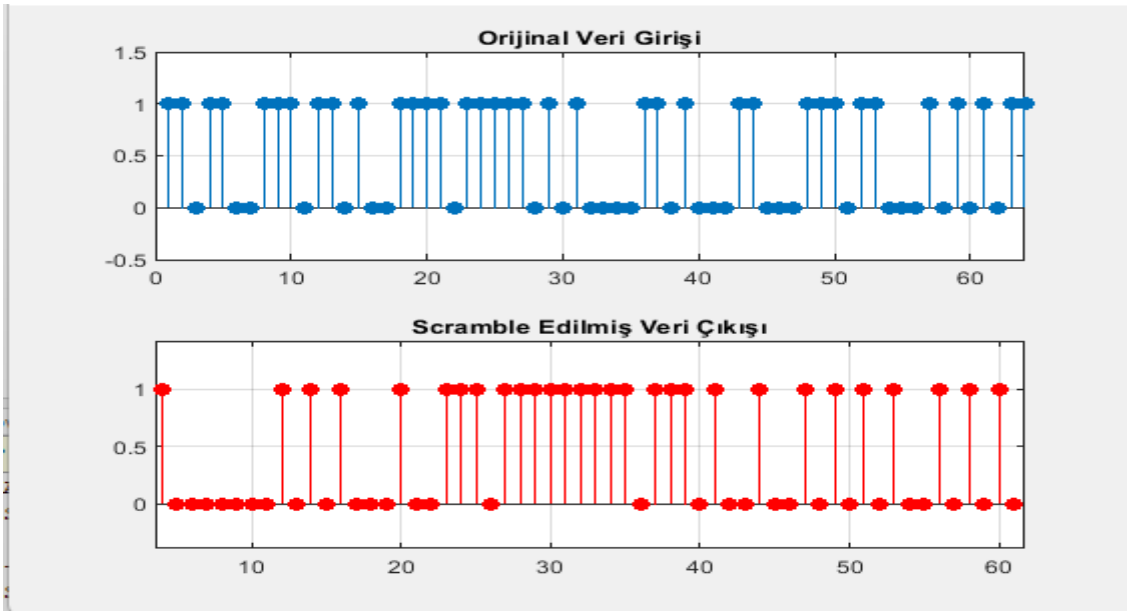
1 1 0 1 1 0 0 1 1 1 0 1 1 0 1 0 0 1 1 1 1 0 1 1 1 1 1 0 1 0 1 0 0 0 0 1 1 0 1 0 0 0 1 1 0 0 0 1 1 1 0 1 1 0 0 0 1 0 1 0 1 0 1 1

ÇIKIŞ - BÖLÜM 1: SERVICE FIELD (İlk 16 Bit - Input hep 0 idi):

0 1 1 0 1 1 0 0 0 0 0 1 1 0 0 1

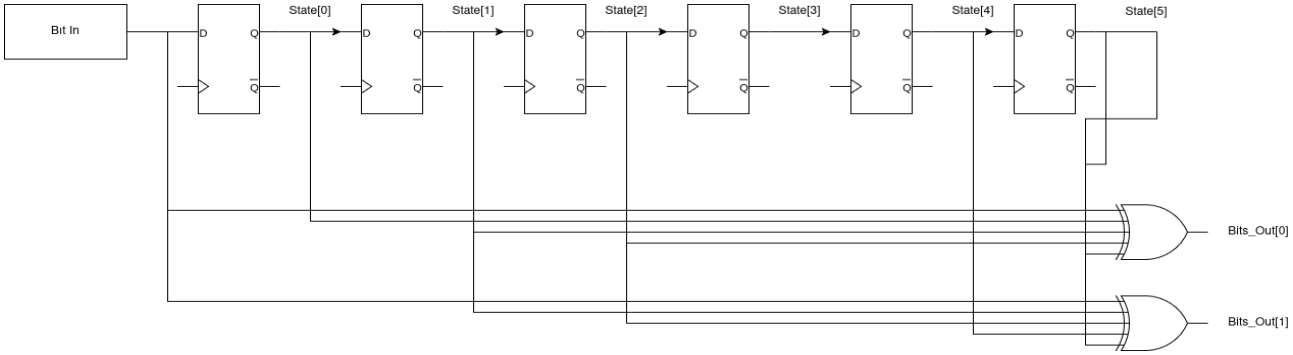
ÇIKIŞ - BÖLÜM 2: SCRAMBLED DATA (Karıştırılmış Veri):

0 1 1 1 0 0 0 0 0 0 0 1 0 1 0 1 0 0 0 1 0 0 1 1 1 0 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 0 0 1 0 0 1 0 1 0 1 0 1 0 1 0 0 1 0 1 0 1 0 1 1 1



Convolutinal Encoder

Convoluttinal Encoder shift register ve belli başlı logic fonksiyonlarından oluşan ve kanal kodlama için kullanılan bir bloktur. Bu blok Punchring bloğu ile birlikte kodlama bloklarını oluştulur. Convolutinal Encoder içinde Gelen bitlere göre yeni 2 bit üreten bir shift register bulunur. Bu kanal kodlama blokları gürültülü ortamda çıkış gücünü artırmadan bit error rate düşürmek için kullanılır.



Bu bitler oluşturuduktan sonra oluşturulan bitler fsm2 ye gönderilmek üzere axi_fifo_brama aktarılır.

MATLAB Modeli

```
function [bits_out] = conv_encoder_80211(bit_in, reset)
   % persistent değişkenler donanımdaki 'reg' (state) yapısını korur
   persistent state;

   % Reset durumunda veya ilk çalıştırmada register'ı sıfırla
   if nargin > 1 && reset || isempty(state)
      state = zeros(1, 6); % 6-bit memory (K=7 için)
   end

   % Çıkış dizisini önceden tanımla (Her giriş için 2 çıkış)
```

```
    bits_out = zeros(1, length(bit_in) * 2);

    for i = 1:length(bit_in)
        curr_bit = bit_in(i);

        % Verilog logic:
        % g0 = 133 (oktal) -> 1 0 1 1 0 1 1 (Binary)
        % g1 = 171 (oktal) -> 1 1 1 1 0 0 1 (Binary)
        % Donanımdaki state[5:0] yapısına göre XOR işlemleri:

        out0 = mod(curr_bit + state(1) + state(2) + state(3) + state(6), 2);
        out1 = mod(curr_bit + state(2) + state(3) + state(5) + state(6), 2);

        % Çıkışları ata
        bits_out(2*i - 1) = out0;
        bits_out(2*i)     = out1;

        % Kaydırmalı Kaydediciyi Güncelle (state <= {state[4:0], bit_in})
        state = [state(2:end), curr_bit];
    end
end
```

Çıktısı:

```
>> Conv_encoder_test
Giriş Bitleri: 1  0  1  1  0  0  0  1  1  1
Kodlanmış Bitler: 1  1  1  1  1  0  0  0  0  1  1  0  0  1  1  1  0  1  1  1
```

Bit_enc_fifo

Bu FIFO, Convolutional Encoder'dan gelen bitleri modülasyon aşamasına hazırlamak için tamponlayarak puncturing (delme) ve hız eşleme işlemlerini koordine etmek amacıyla kullanılır. Yazma işlemi (input), paket iletimi başladığı andan itibaren tüm veri ve dolgu bitleri bitene kadar (`state11 < S11_RESET`) aktif kalarak kodlanmış bitleri sürekli içeri alır. Okuma işlemi (output) ise sadece IQ örnekleme aşamasında (`S2_PUNC_INTERLV`), o anki bitin işlenmesi tamamlandığında veya puncturing mantığına göre yeni bit talep edildiğinde aktifleşerek veriyi bir sonraki bloğa aktarır.

Bit Puncturing and Bit Interleaving

Convolutional Encoder ve FIFO aşamasından sonra gelen bu blok, verinin iletim hızına (Data Rate) göre düzenlenmesini ve kablosuz kanalın bozucu etkilerine karşı direnç kazanmasını sağlar. 802.11 standardında tanımlanan farklı veri hızlarına ulaşmak için kodlama oranının (Coding Rate) değiştirilmesi gerekir; bu işlem **Puncturing** (Delme) ile yapılır. Ardışık bit hatalarını (Burst Errors) dağıtmak için ise **Interleaving** (Serpiştirme) uygulanır.

OpenOFDM tasarımında bu karmaşık matematiksel işlemler, donanım yükünü hafifletmek amacıyla önceden hesaplanmış bir tablo (LUT - Look Up Table) üzerinden yürütülür.

punc_interlv_lut (Referans Tablosu)

Bu modül, FSM 2'nin beyni gibi çalışır. Standartta belirtilen karmaşık permütasyon formüllerini hesaplamak yerine, her bir bitin gitmesi gereken adresi ve delinip delinmeyeceği bilgisini doğrudan verir.

- **Girişler:** `rate` (Veri hızı, örn: 6 Mbps, 54 Mbps) ve `idx_i` (Gelen bitin sırasını sayan sayaç).
- **Çıkışlar:** `idx_o` (Bitin RAM'de yazılacağı adres) ve `punc_o` (Delme bayrağı).

Eğer `punc_o` çıkışı '1' ise, o bitin iletilmemesi gerektiği anlaşılır ve bit sistemden atılır.

Sistem `S2_PUNC_INTERLV` durumundayken, `bits_enc_fifo`'dan kodlanmış verileri çeker ve `bits_ram`'e yazar. Ancak bu yazma işlemi sıradan bir kopyalama değildir; Puncturing ve Interleaving işlemleri eş zamanlı olarak burada gerçekleşir.

1. **Veri Seçimi ve Puncturing:** FIFO'dan her saat çevriminde 2 bit (`bits_enc_fifo_odata[1:0]`) gelir. Ancak modülasyona tek tek bitler gitmelidir. `enc_pos` adı verilen bir yazmaç, bu iki bitten hangisinin işleneceğini takip eder.
    - Eğer LUT modülü `punc_o` sinyalini **0** gönderirse (Delme yok), ilgili bit seçilir ve RAM'e yazılır.
    - Eğer `punc_o` sinyali **1** gelirse (Delme var), donanım bu biti görmezden gelir. Multiplexer (seçici), o biti atlayarak diğer biti seçer veya yazma işlemini pas geçer. Böylece bit fiziksel olarak silinmek yerine, RAM'e hiç yazılmayarak "delinmiş" olur.
2. **Adresleme ve Interleaving:** `bits_ram`, modülasyon öncesi bir tampon görevi görür. Veriler buraya yazılırken sıralı (0, 1, 2...) değil, LUT'tan gelen **karışık adreslere** (`interlv_addrs`) göre yazılır. Örneğin; gelen 1. bit 0. adrese yazılırken, 2. bit 48. adrese yazılabilir. Bu karıştırma işlemi **Interleaving**'dir.

- RAM tamamen dolduğunda (`dbps_cnt_FSM2` sayacı dolduğunda), sistem `S2_MOD_IFFT_INPUT` durumuna geçer. Bu aşamada veriler RAM'den sırasıyla okunarak Modülasyon bloğuna (BPSK, QPSK, 16-QAM vb.) gönderilir. Yazarken karıştırıp okurken sıralı okuduğumuz için, havaya çıkan verilerin sırası harmanlanmış olur. Bu sayede kanaldaki anlık bir sönümleme (fading), peş peşe gelen bitlerin hepsini bozmaz; hata düzeltme kodunun (FEC) kurtarabileceği şekilde hataları zamana yayar. MATLAB Modeli ve Çıktısı çok uzun olduğu için eklenememiştir. Githuba yüklenicek.

IFFT, Modulation Ve Piloting Blokları

802.11n Standartında OFDM tekniği kullanılır. OFDM(Orthoganal Frequency Division Multiplexing) kablosuz haberleşmede kullanılan ve iletilen bilginin birden fazla birbirine dik

(Spektral Düzlemde) SubCarier ile taşınmasıdır. 802.11n Standartında 64 adet Subcarrier kullanılır. İletim tipi HT ise bu Subcarierların dağılımı şu şekildedir:

`iq_cnt` (Index)	Frequency Index	Type	Description
0	0	**Null**	DC Component
1 - 6	+1 to +6	**Data**	Positive Data Subcarriers
7	+7	**Pilot**	Pilot 2 (`pilot_iq[2]`)
8 - 20	+8 to +20	**Data**	Positive Data Subcarriers
21	+21	**Pilot**	Pilot 3 (`pilot_iq[3]`)
22 - 28	+22 to +28	**Data**	Positive Data Subcarriers (**Extended range**)
29 - 35	+29 to -29	**Null**	**Guard Band**. Narrower than legacy. Includes Nyquist (-32) at index 32.
36 - 42	-28 to -22	**Data**	Negative Data Subcarriers (**Extended range**)
43	-21	**Pilot**	Pilot 0 (`pilot_iq[0]`)
44 - 56	-20 to -8	**Data**	Negative Data Subcarriers
57	-7	**Pilot**	Pilot 1 (`pilot_iq[1]`)
58 - 63	-6 to -1	**Data**	Negative Data Subcarriers

Bu tabloda görüldüğü üzere OFDM subcarierları pilot, boşluk ve Data olarak ayrılmıştır.

PILOT, DC (0Hz) and sideband(SB)

Pilot subcarierları **Common Phase Error** (Faz Kayması) ,**Residual Frequency Offset** (iletim sırasında yaşanılabilecek frekans kaymaları) ve **Sampling Frequency Offset** (Alıcı ve Vericinin Sample Ratelerin farklı olmasından kaynaklı hatalar) önlemek için kullanılır. Bu Pilotlar içerden Pilot Scrambler adı verilen bir LFSR ile işleme girer. Bu işlem sonucunda Pilot sinyallerin fazları (IQ değerleri) değişir. Bu işlem Spectral çizgilerin oluşmasını engellemek için gerçekleşir.
Pilot Scrambler LFSR böyle verilmiştir(pilot_scram_state değeri dışarıdan verilir(seed)):

```
pilot_scram_state <= {pilot_scram_state[5:0], (pilot_scram_state[3] ^ pilot_scram_state[6])};
```

Bu LFSR'nin çıktısı plot_gain olarak şu şekilde tanımlanmıştır:

```
wire pilot_gain = pilot_scram_state[6] ^ pilot_scram_state[3] ^ 0;
```

Sonrasında ise HT sinyaller için Pilot IQ'leri ekstrandan her clock cycleda dönecek şekilde ayarlanmıştır ve bu işlem için ht_polarity adında değişken tanımlanmıştır. Bu değişkenle beraber Plot Subcarierları şu şekilde tanımlanmıştır:

```
if(PKT_TYPE == HT && ofdm_cnt_FSM2 > 2) begin

if(ht_polarity[0] ^ pilot_gain == 0)

pilot_iq[0] <= {16'h4000, 16'h0000};
else
pilot_iq[0] <= {16'hC000, 16'h0000};
if(ht_polarity[1] ^ pilot_gain == 0)
pilot_iq[1] <= {16'h4000, 16'h0000};
else
pilot_iq[1] <= {16'hC000, 16'h0000};
if(ht_polarity[2] ^ pilot_gain == 0)
pilot_iq[2] <= {16'h4000, 16'h0000};
else
pilot_iq[2] <= {16'hC000, 16'h0000};
if(ht_polarity[3] ^ pilot_gain == 0)
pilot_iq[3] <= {16'h4000, 16'h0000};
else
pilot_iq[3] <= {16'hC000, 16'h0000};
ht_polarity <= {ht_polarity[0], ht_polarity[3:1]};
```

Tabloda verildiği üzere DC ve en uç bantlar boş bırakılır. DC kısmının boş bırakılmasının nedeni osilatör sızıntılarını önlemekken uç bantlar diğer sinyallerle karışmasını önlemek için boş bırakılır.

Modülasyon

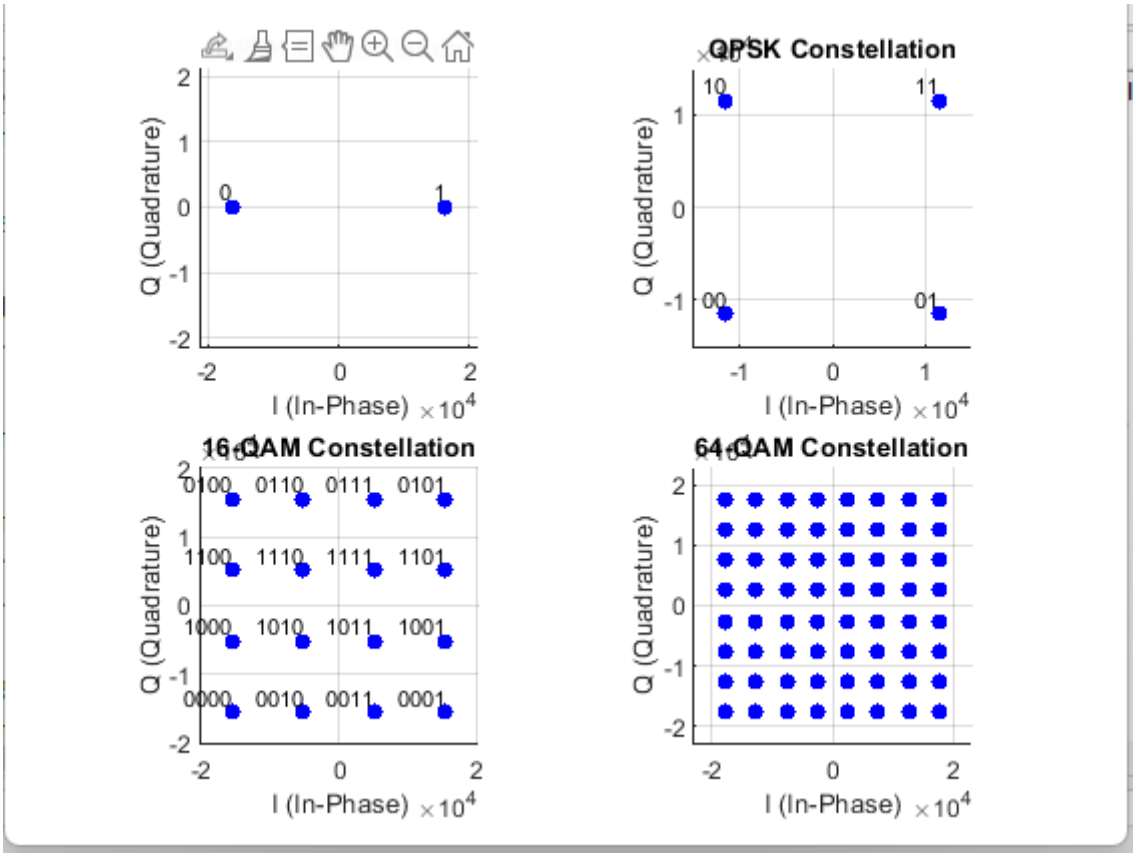
HT sinyaller için 52 adet data Subcarier ayrılmıştır. Bu data subcarierların kaç bit taşıyacağı ve bu bitlerin hangi modülasyondan geçiceği headerdaki MCS indexi ile verilmiştir. Bu Tx modülünde kullanılabilecek 4 adet modülasyon tipi bulunur. Bunlar:

- BPSK(Binary Phase Shift Keying) Subcarier başına düşen bit sayısı =1
- QPSK(Quadrative Phase Shift Keying) Subcarier başına düşen bit sayısı =2
- 16-QAM (16 Quadrature amplitude modulation) Subcarier başına düşen bit sayısı =4
- 64-QAM (64 Quadrature amplitude modulation) Subcarier başına düşen bit sayısı = 6

Hangisinin Kullanılacağı FSM1deki çözülen N_BPSC değişkeni ile verilir. Modülün içerisende puncturing/interleave bloğunun çıkışı olan SIMO'dan modüle edicek bitleri alır. (Modüle edilecek bitler Puncturing/interleave bloğu tarafaından gerekli şekilde ayrılıyor SIMO içerisinde) Modülasyon bloğu bitleri alır ve onlara IQ değeri atar. Bu IQ değerleri sonrasında Data için ayrılan Subcarier indexine gelindiğinde IFFT bloğuna aktarılır.

MATLAB modeli git’e yüklenicek.

Çıktısı

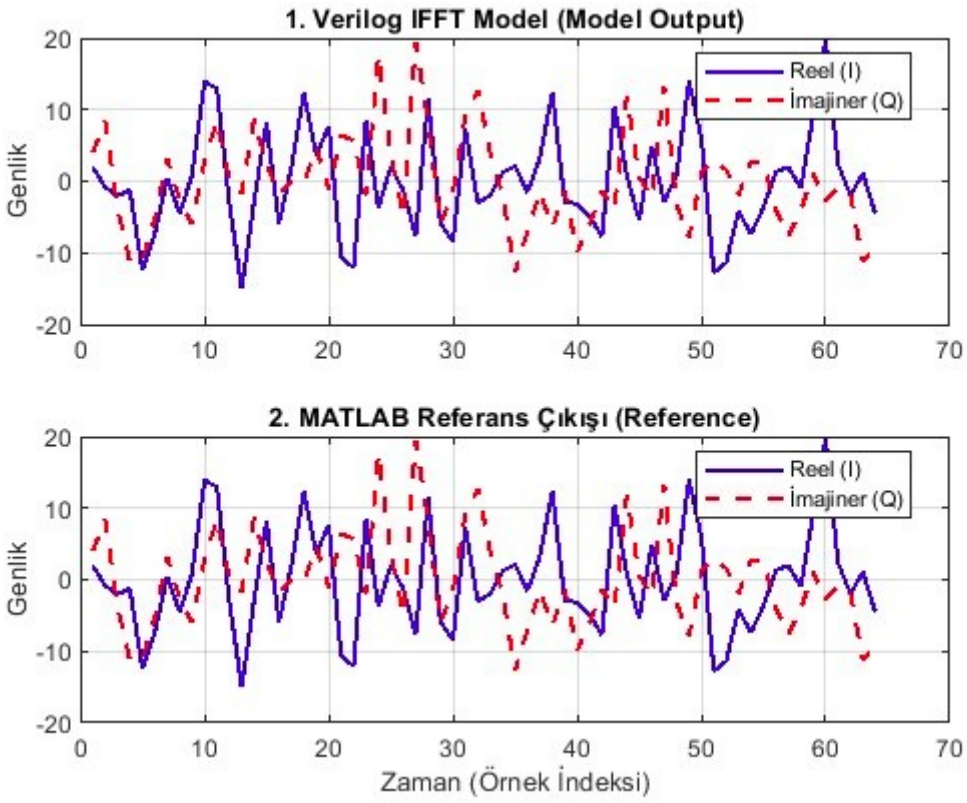


IFFT

IFFT bloğu her clock cycleda 1 frekans domain örneği alırken önceki sembolün time domain çıkışlarını verir. Bu blok S2_MOD_IFFT_INPUT durumunda ilk örneklerini almaya başlar. Oluşturulan semboller 64 adet subcarier içerir ve blok bu 64 subcarierdan 64 adet time domain sample verir. Bu samplelar ilk index dc , 32.indexe kadar pozitif sonrasındakiler ise negatif subcarier olarak geçer. (Bu durumda frekans domainde f=0 sağ tarafı ile sol tarafı eşit olmadığı için elde ettiğimiz time domainler complex sinyallerdir. Bu durumda zaman domain gerçel kısmı In Phase carier ile gönderilirken imajiner kısmı Quadrature carier ile gönderilir.) Bu blok ilk 64 frekans samplenı (1 sembol) aldıktan sonra sırasıyla her clock cycle da time domain çıktılarını verir. 2. sembolün frekans çıktıları almayı bitirdekten sonra 3. sembolün frekans çıktılarını almaya başlar bu sırada 2. sembolün time domain çıktılarını verir. Bu durum gönderecek sinyal kalmayana kadar devam eder.

MATLAB Modeli Çıktısı:

(Data değerleri rastgele verilmiştir)



Cyclic Prefix (CP)

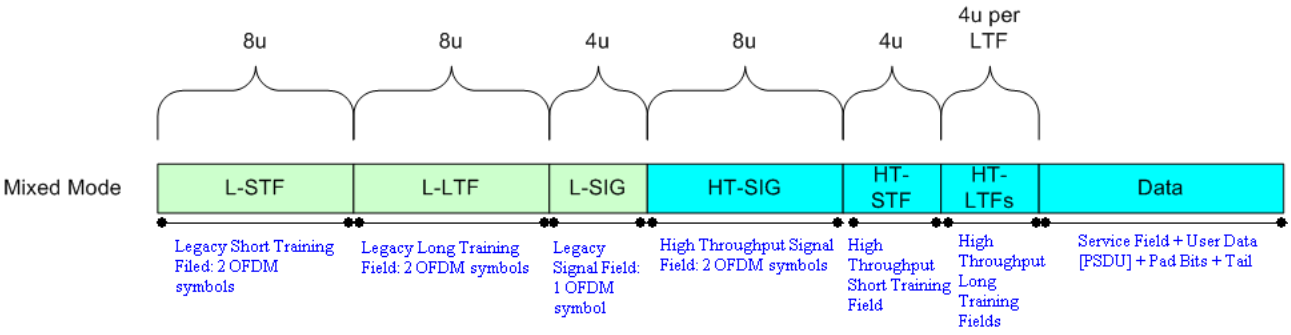
Cyclic Prefix (CP), bir OFDM sembolünün son kısmının kopyalanıp aynı sembolün başına eklenmesi işlemidir. Bu yöntem, kablosuz iletimde çoklu yol yayılımından kaynaklanan gecikmeleri tolere eden bir koruma aralığı oluşturarak Semboller Arası Girişimi (ISI) engeller. Ayrıca, bu kopyalama işlemi lineer konvolüsyonu dairesel konvolüsyona çevirdiği için alt taşıyıcıların dikliğini (ortogonalliğini) korur ve alıcı tarafında kanal denkleştirmeyi (equalization) mümkün kılar.

CP fifo & Pkt Fifo

Pkt fifo Time domaindeki oluşturulan Tüm OFDM sembollerini depolar. CP fifo ise verilen Guard Interval süresine bağlı olarak sondaki 16’lık (800ns) veya 8’lik (400ns) OFDM sembollerini paketler. Bu paketlemenin yapılma sebebi sondaki sembollerin başa eklenerek sembol süresini uzatıp koruyucu bir süre eklemektir. Bu FIFOlara kaydedilen semboller S3_DATA durumunda gönderilmeye başlar.

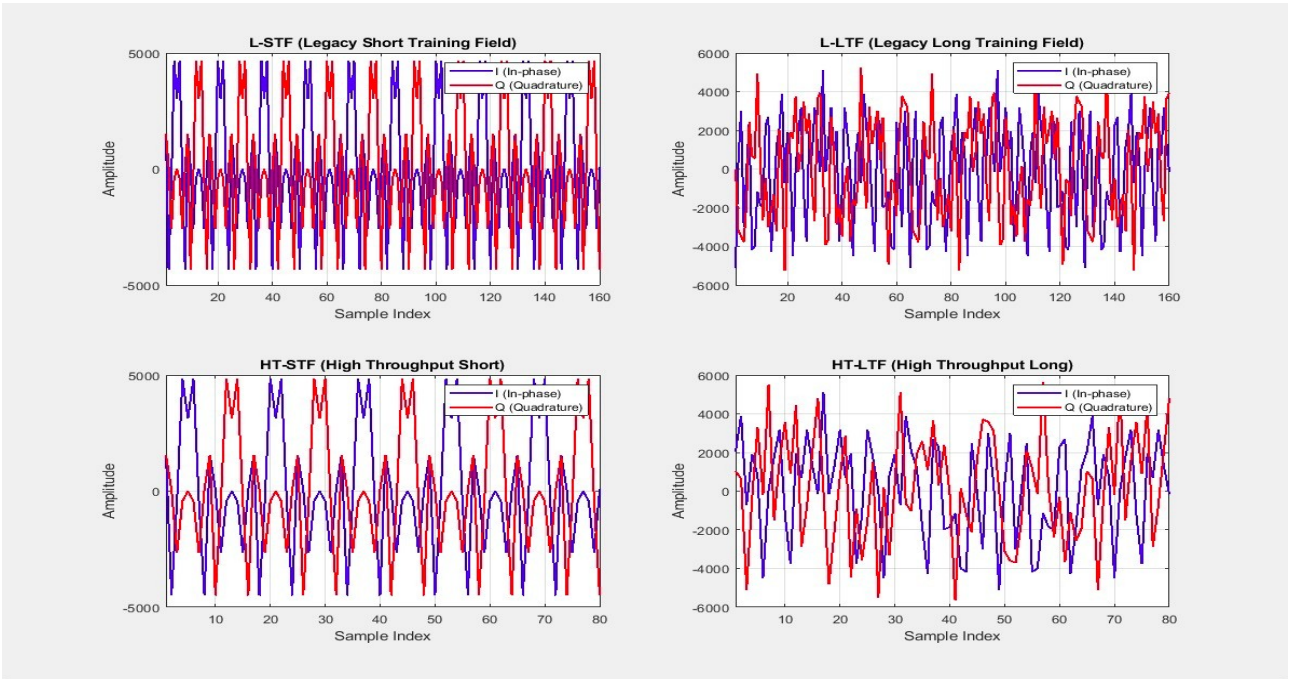
FSM3 & Ve Preamblelar

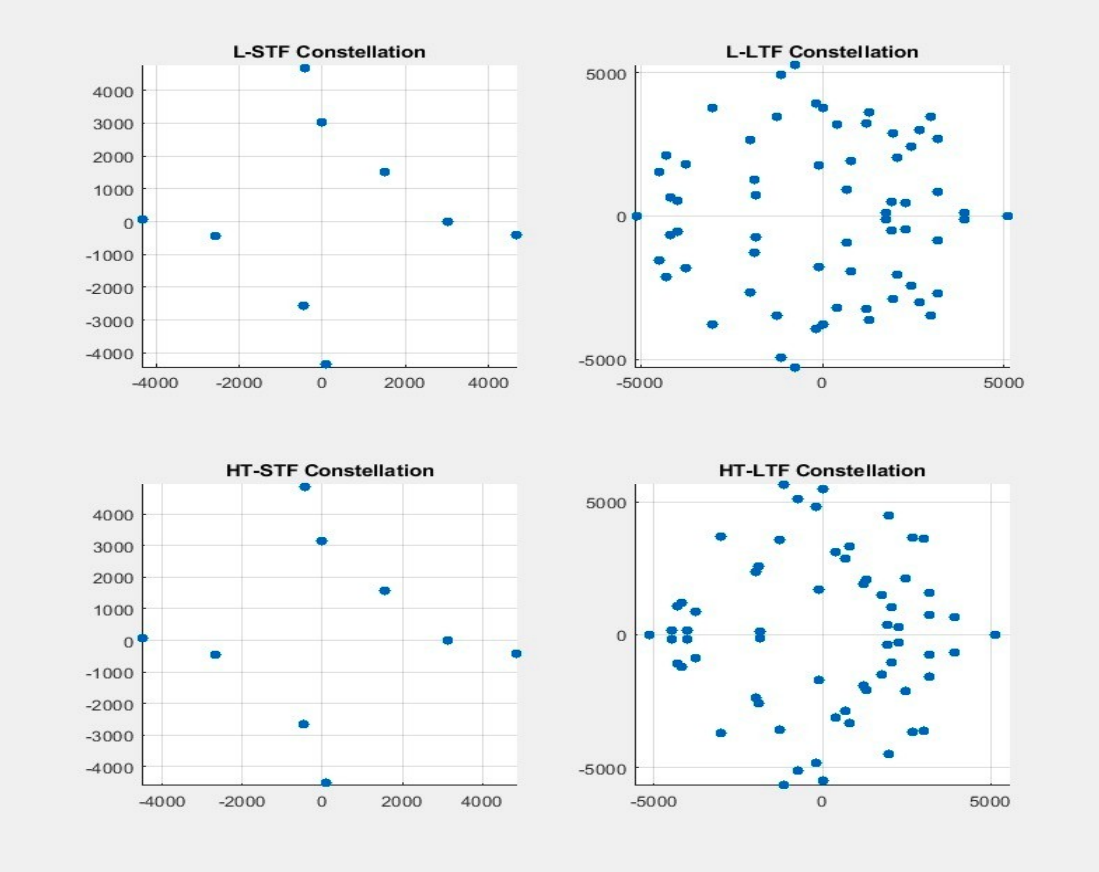
Oluşturduğumuz OFDM sembollerinin alıcı tarafının sinyali doğru bir şekilde işleyebilmesi için başına başlık verisi(preamble) eklenir. Bu başlıklar L_STF , L_LTF, L_SIG, HT_SIG, HT_STF, HT_LTF şeklinde ayrılmıştır.



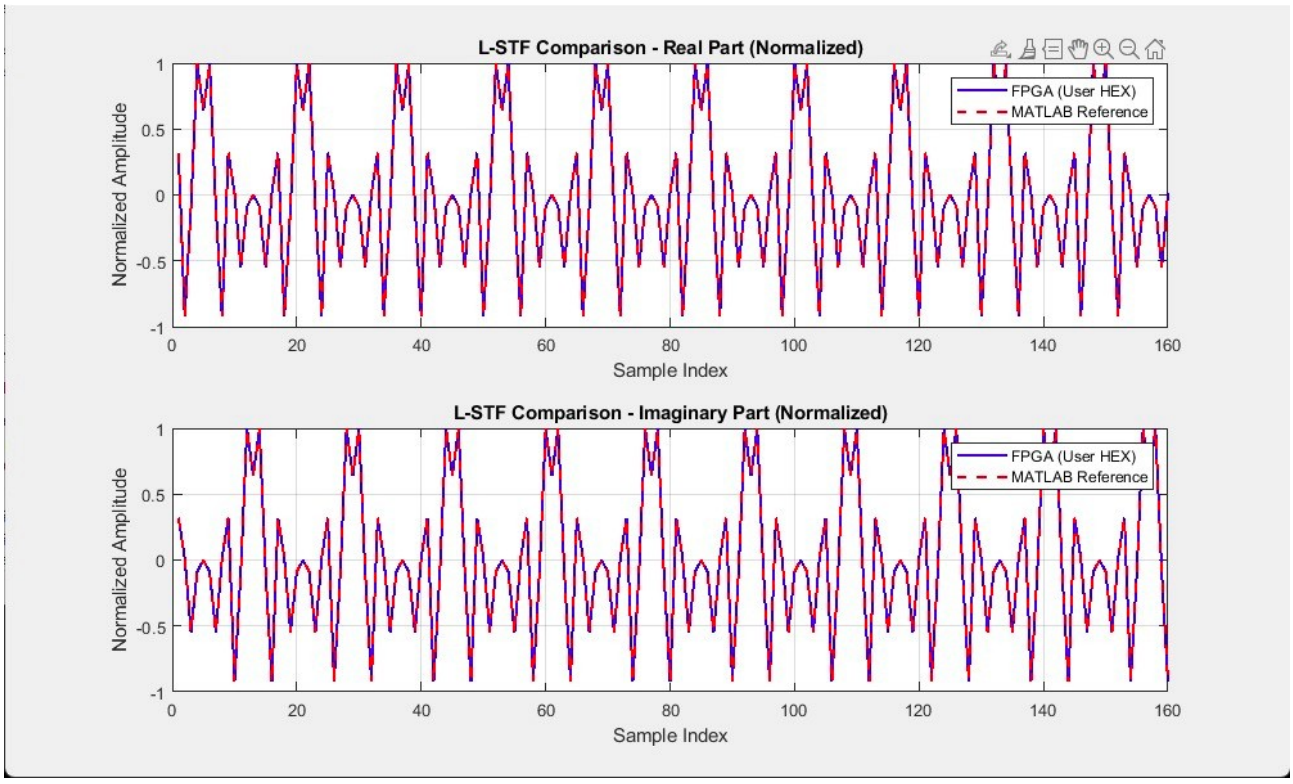
L_STF ve L_LTF alıcının kanal hesaplaması için kullandığı sabit IQ verileridir. L_STF ve L_LTF legacy cihazların yanı sıra HT prosedüründe de kullanılır. Sonrasında headerdaki L_SIG ofdm sembolüne dönüştürülüp gönderilir. Sonrasında HT_SIG header gönderilir. Headerdan bağımsız olarak alıcının HT kanalı için veri almaya hazırlanması ve kanal tahmini için kulanılan HT_STF ve HT-LTF sinyalleri gönderir. Bunlar ROM dan hazır olarak okunur ve her paket için sabittir.

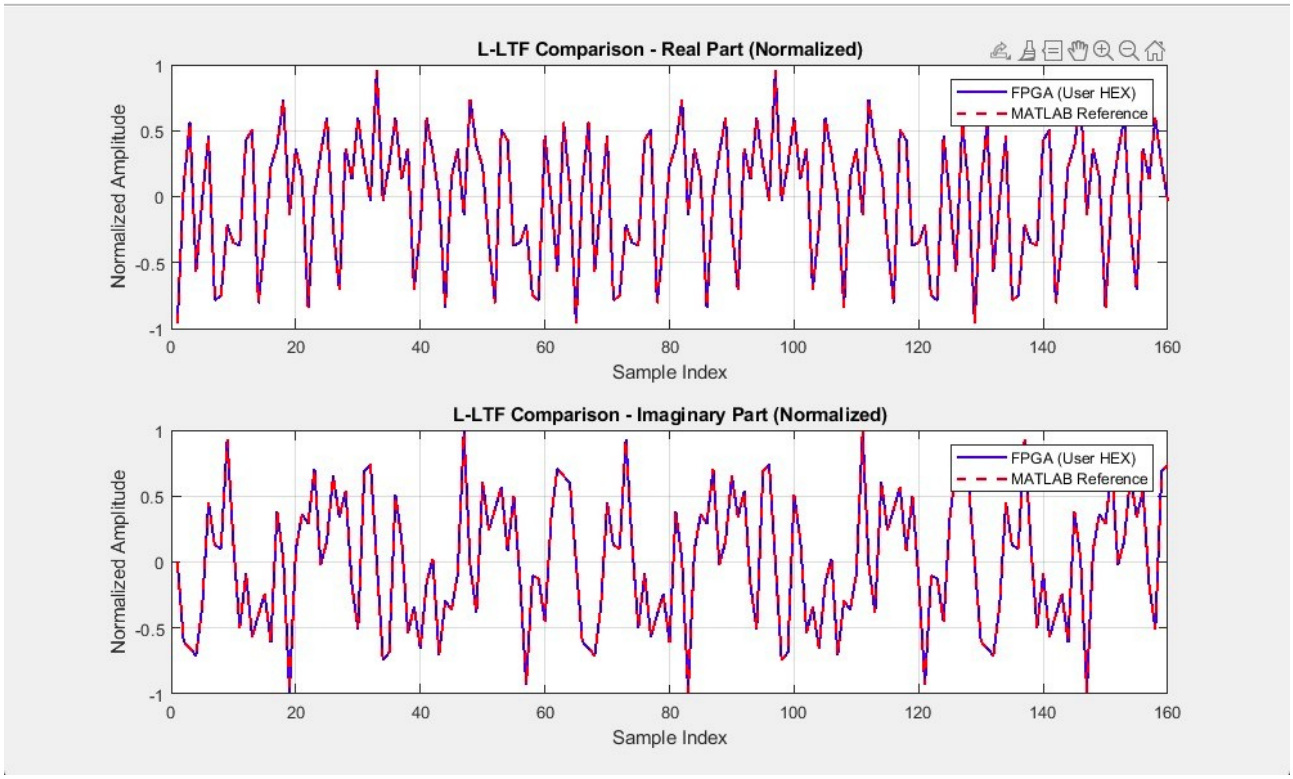
ROM’daki Değerlerin MATLAB Çizimi ve IQ dağılımı

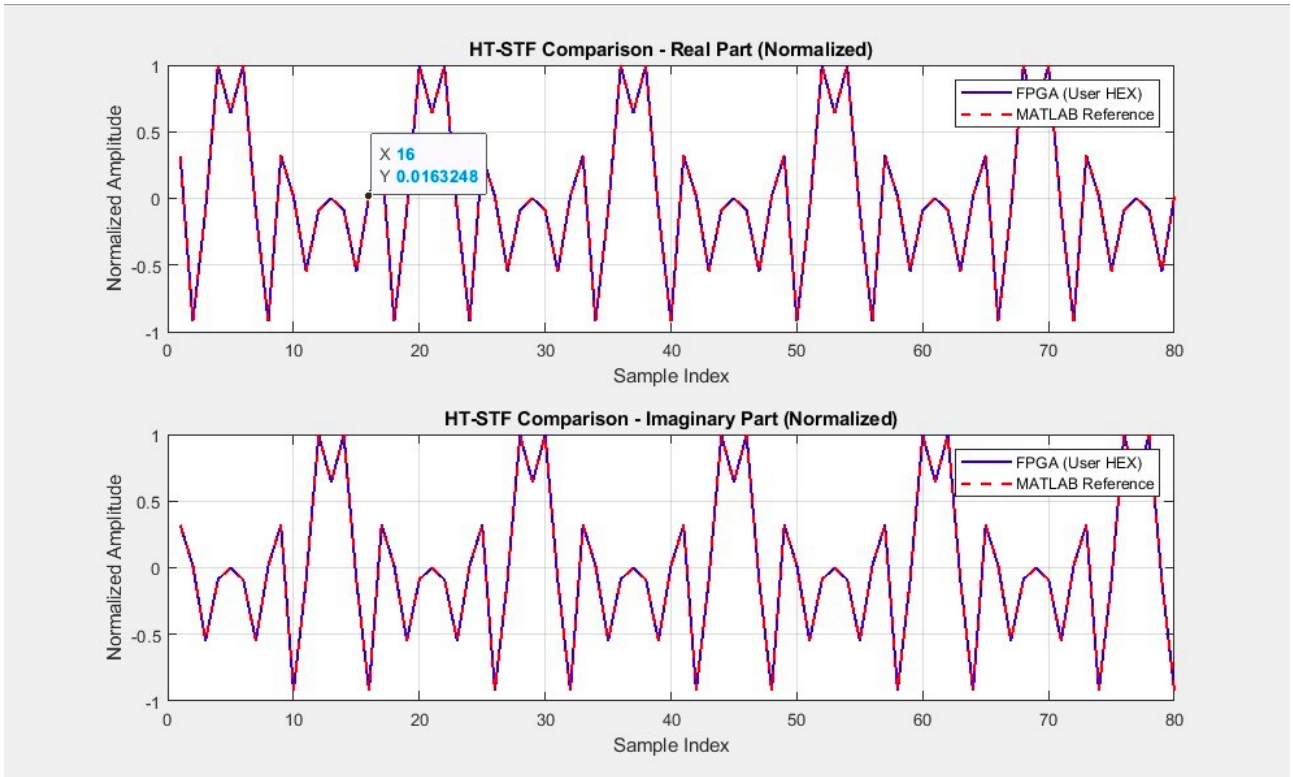


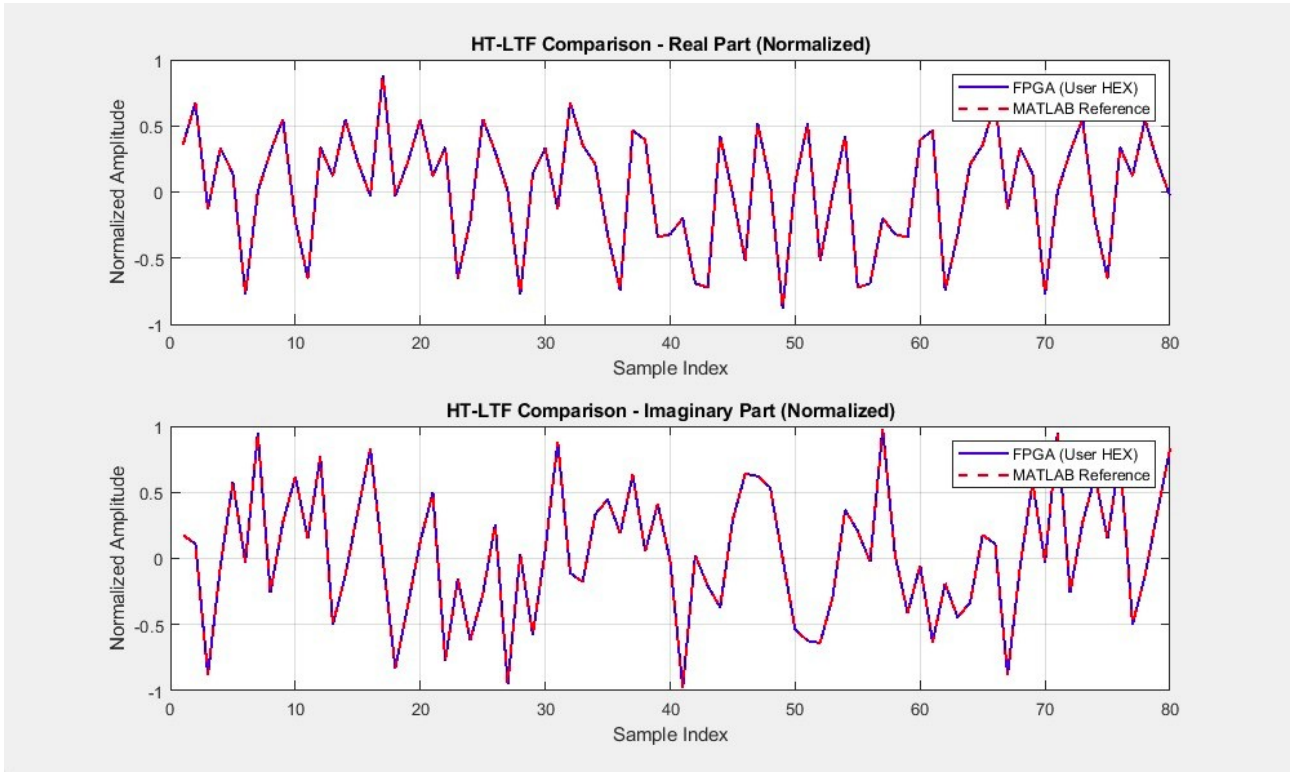


MATLAB WLAN Toolbox'un Altın Standart Sinyalleri ile karşılaştırılması:

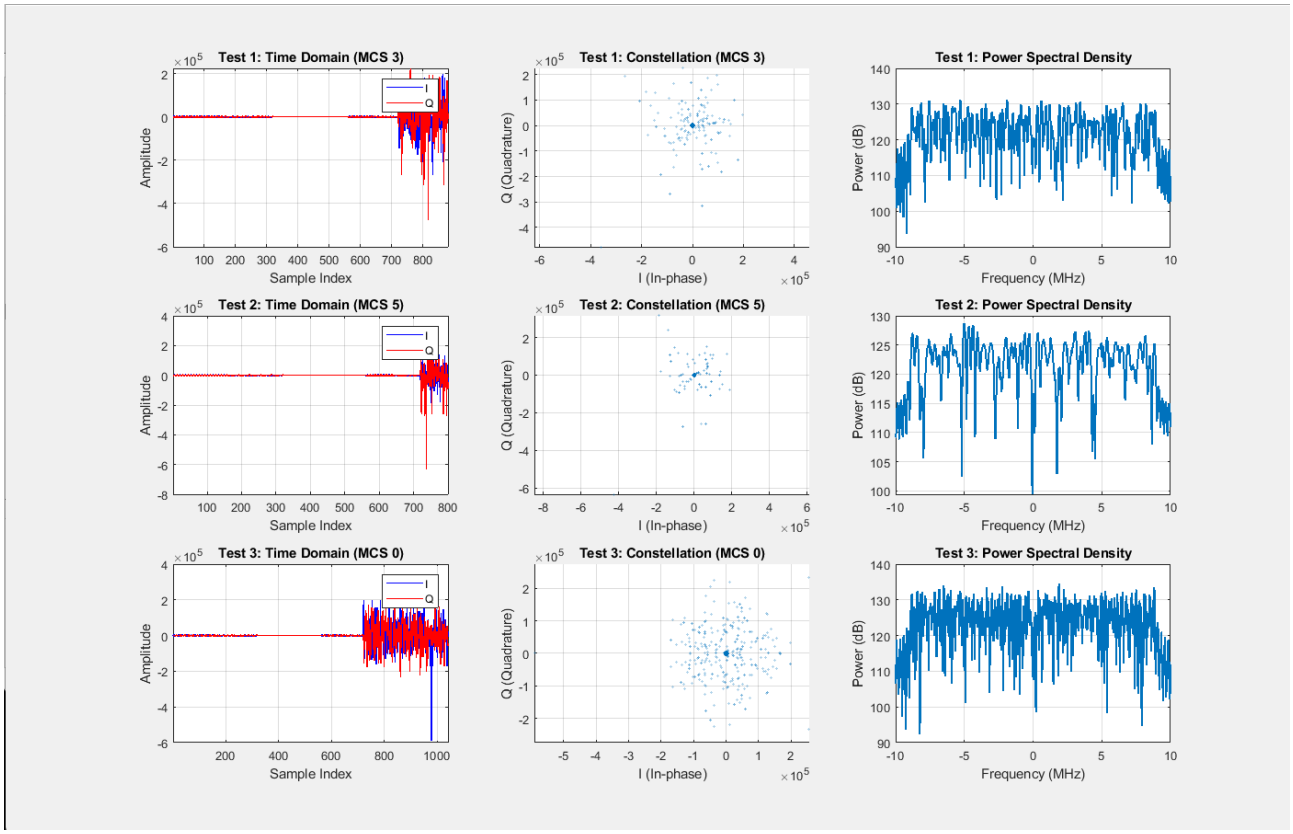



L-LTF Comparison - Real Part (Normalized)
FPGA (User HEX)
MATLAB Reference
Normalized Amplitude
Sample Index
L-LTF Comparison - Imaginary Part (Normalized)
FPGA (User HEX)
MATLAB Reference
Normalized Amplitude
Sample Index


HT-STF Comparison - Real Part (Normalized)
X 16
Y 0.0163248
FPGA (User HEX)
MATLAB Reference
Normalized Amplitude
Sample Index
HT-STF Comparison - Imaginary Part (Normalized)
FPGA (User HEX)
MATLAB Reference
Normalized Amplitude
Sample Index



Tx Modeli MATLAB Sonuçları:



Test 1 MCS3 : 16 QAM ½ rate

Data : “DEADBEEFCAFEBABE1234567890ABCDEF”

Test 2 MCS5: 64 QAM 2/3 rate

Data: “0123456789ABCDEF”

Test 3 MCS 0: BPSK ½ rate

Data: “FFFFFFFFFFFFFFFF”

test_hex_to_ofdm_ht dosyası üzerinden farklı coding rate,modülasyon teknikleri ve datalar test edilebilir.